

Funciones hash

y sus aplicaciones

Una huella de 256 bits para cualquier cosa que exista: un nombre, una película, una biblioteca entera. De ese único truco cuelgan la firma digital, el árbol de Merkle, el compromiso, el minado y la inmutabilidad de la cadena. Aquí lo desarmamos pieza por pieza.



Índice

1. Qué hace exactamente una función hash	2
1.1. Tres cosas que la licuadora cumple	2
1.2. La consecuencia incómoda: las colisiones existen	2
2. Las tres resistencias, y por qué no son la misma	3
3. Cómo está construida por dentro, y qué se rompe por eso	5
3.1. El ataque de extensión de longitud	6
3.2. La construcción correcta: HMAC	6
4. El ataque del cumpleaños: por qué 256 bits valen 128	7
4.1. Derivación de la cota	7
5. Cuáles usar hoy, cuáles ya cayeron, y cuándo	8
6. Contraseñas: por qué un hash rápido es un defecto	9
6.1. Los tres escenarios	10
7. Árboles de Merkle: probar pertenencia sin descargar todo	11
8. Compromisos, prueba de trabajo y la cadena	12
8.1. Minar es resolver una preimagen parcial	13
9. Síntesis: qué se rompe exactamente cuando falla cada propiedad	13
9.1. Una nota sobre el adversario cuántico	14
A. Resumen de cotas	16
B. Ejercicios propuestos	16

1. Qué hace exactamente una función hash

Objetivo del módulo

Reconocer las cuatro propiedades observables de una función hash —determinismo, longitud fija, uniformidad y avalancha— y distinguir un hash criptográfico de uno que no lo es.

Piensa en una licuadora industrial. Metes lo que sea —una palabra, una película, la Biblioteca Nacional— y siempre sale exactamente un litro de puré. Siempre un litro: ni más si metes más, ni menos si metes menos. Y el puré de una manzana nunca se parece al puré de dos manzanas: no hay «casi igual».

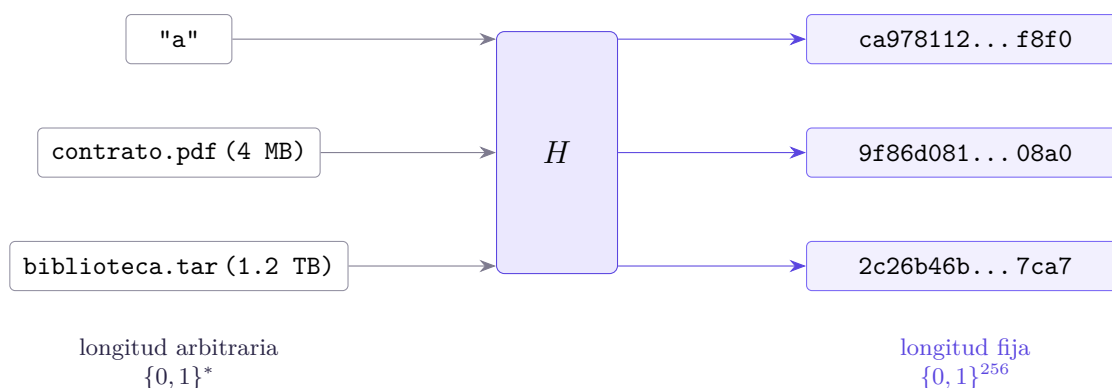


Figura 1: Una función hash aplasta un dominio infinito sobre un codominio finito. Ésa es su utilidad y, a la vez, la razón por la que las colisiones son inevitables.

1.1. Tres cosas que la licuadora cumple

Es determinista. La misma manzana, el mismo puré, hoy y en diez años, en tu máquina y en la mía.

No se puede des-licuar. Del puré no recuperas la manzana.

Es discontinua a propósito. Quitarle un pixel a una foto no cambia «un poquito» el resultado: lo cambia por completo. Eso es el *efecto avalancha*, y es la propiedad que vuelve útil a un hash para detectar alteraciones.

La figura 2 muestra esa última propiedad sin metáforas. Son los 256 bits de dos digests reales que difieren en una sola letra de la entrada. No hay nada de mágico en ello: es el resultado buscado de sesenta y cuatro rondas de sumas, rotaciones y XOR.

1.2. La consecuencia incómoda: las colisiones existen

Las entradas posibles son infinitas y las salidas son 2^{256} . Por el principio del palomar, hay infinitos pares de mensajes distintos con el mismo hash. La criptografía no los elimina —no puede—: sólo los vuelve *inencontrables*.

Definición 1 (Función hash criptográfica). Una función hash es una aplicación

$$H : \{0,1\}^* \longrightarrow \{0,1\}^n$$

de cadenas de longitud arbitraria a cadenas de longitud fija n , a la que se le exige:

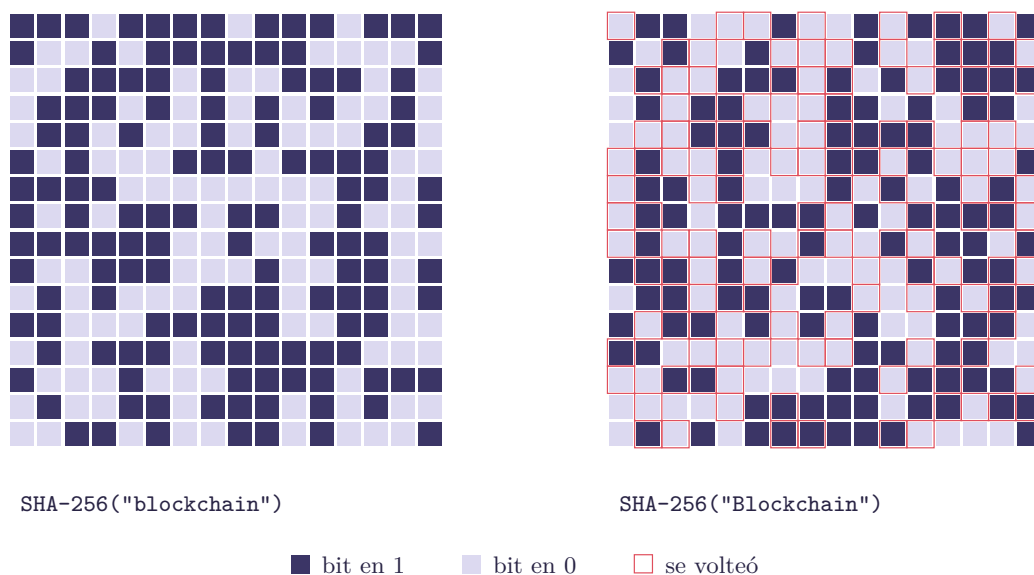


Figura 2: Espejo de bits. Cambiar una mayúscula voltea **130 de los 256 bits**: el 50.8 %. Las celdas marcadas en rojo son las que cambiaron.

- (I) **Eficiencia.** Calcular $H(x)$ cuesta poco: lineal en $|x|$ en la práctica.
- (II) **Uniformidad.** Para x al azar, $H(x)$ se distribuye aproximadamente uniforme sobre $\{0, 1\}^n$. El modelo idealizado se llama *oráculo aleatorio*.
- (III) **Avalancha.** Si x y x' difieren en un bit, cada bit de $H(x) \oplus H(x')$ vale 1 con probabilidad $\approx \frac{1}{2}$, de modo que

$$\mathbb{E}[d_H(H(x), H(x'))] \approx \frac{n}{2}.$$

Laboratorio 1.1 — Medidor de avalancha

El 50 % no basta, y CRC32 lo demuestra. El experimento toma 400 mensajes aleatorios, les voltea siempre el mismo bit de entrada y mide la distancia de Hamming entre los dos digests, expresada como fracción del digest.

Qué observar. CRC32 da un promedio de aspecto respetable, cerca del 50 %, y aun así produce *un solo patrón de diferencia* con desviación $\sigma = 0$: voltear ese bit cambia siempre exactamente los mismos bits de salida, sin importar el mensaje. Eso no es mezcla, es una ecuación que el atacante puede resolver. SHA-256 produce 400 patrones distintos en 400 pruebas y una σ cercana a $\sqrt{n}/2$. *El promedio miente; los patrones no.*

2. Las tres resistencias, y por qué no son la misma

Tres preguntas distintas, tres dificultades distintas. Lo único que las separa es qué recibe el adversario y qué queda libre para que él lo elija.

La consecuencia que hay que memorizar

MD5 y SHA-1 están rotas *en colisión*, no en preimagen. Siguen siendo irreversibles; lo que ya no pueden hacer es garantizar que dos documentos distintos tengan huellas distintas. Por eso una contraseña vieja en SHA-1 no se recupera de golpe, pero una firma sobre SHA-1 no vale nada.

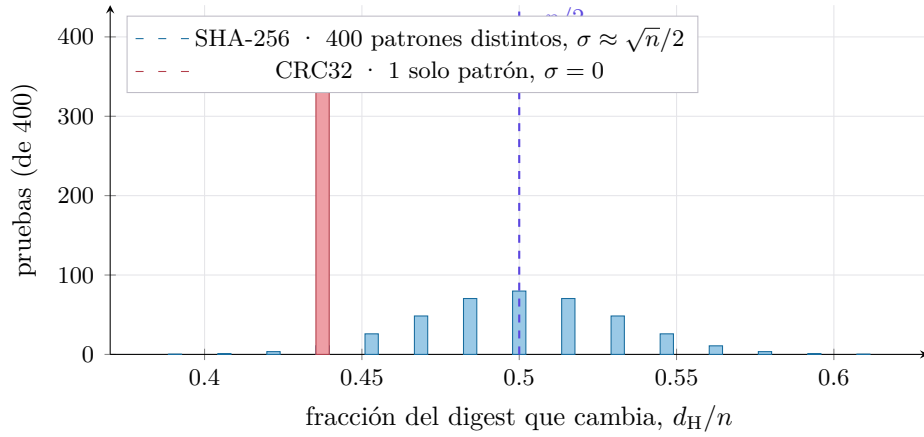


Figura 3: Distribución de la avalancha. Ambas funciones promedian cerca del 50%; sólo una de las dos *mezcla*. La campana es el sello de una moneda lanzada por cada bit de salida; la barra única, el de una función lineal.



Figura 4: Las tres resistencias. Gris lleno = lo que el adversario recibe; contorno punteado = lo que puede elegir. Cuanta más libertad, más barato el ataque.

Proposición 1. Si H es resistente a colisiones, entonces es resistente a segunda preimagen.

Demostración (por contraposición). Supongamos que existe un algoritmo eficiente $\mathcal{A}$ que, dado cualquier x_i , devuelve $x_j \neq x_i$ con $H(x_i) = H(x_j)$. Elegimos x_i al azar y ejecutamos $\mathcal{A}(x_i)$. El par (x_i, x_j) es, por definición, una colisión hallada eficientemente. Luego H no es resistente a colisiones. $\square$

Observación. Al romperse la resistencia a colisión todavía puede sobrevivir la de segunda preimagen —es el caso de SHA-1 hoy—, pero nunca al revés.

Proposición 2. El recíproco falla: existe h resistente a colisiones que no es resistente a preimagen.

Demostración. Sea g resistente a colisiones con salida de n bits. Definimos

$$h(x) = \begin{cases} 1 \parallel x, & \text{si } |x| = n, \\ 0 \parallel g(x), & \text{en otro caso.} \end{cases}$$

Es resistente a colisiones. Si $h(x) = h(y)$, ambas salidas empiezan igual. Si empiezan con 1 entonces $x = y$, contradicción; si empiezan con 0 entonces g colisiona, contradicción.

No es resistente a preimagen. Si y empieza con 1, la preimagen se lee directamente: es el resto de y . $\square$

Moraleja de diseño

Las tres propiedades deben pedirse por separado. La única implicación gratuita es la de la Proposición 1.

Laboratorio 2.1 — Caza de ataques sobre un hash truncado

Con SHA-256 completo nadie encontrará una colisión, hoy ni en un millón de años de cómputo. La seguridad no es magia: **es tamaño**. Truncando el digest a n bits el ataque cabe en un navegador.

Qué observar. La colisión aparece cerca de $2^{n/2}$ intentos; la preimagen exige 2^n . Con $n = 24$ son 4 mil frente a 16 millones. Ésa es toda la diferencia entre este módulo y el del cumpleaños.

3. Cómo está construida por dentro, y qué se rompe por eso

Objetivo del módulo

Entender la construcción iterada de Damgård–Merkle y descubrir el ataque de extensión de longitud, que es la razón de existir de HMAC.

Un hash no procesa el mensaje completo de golpe: lo corta en bloques de 64 bytes y los va digiriendo uno tras otro. Cada bloque se mezcla con el resultado del anterior, como una masa que se amasa por tandas. El estado final, después del último bloque, es el digest.

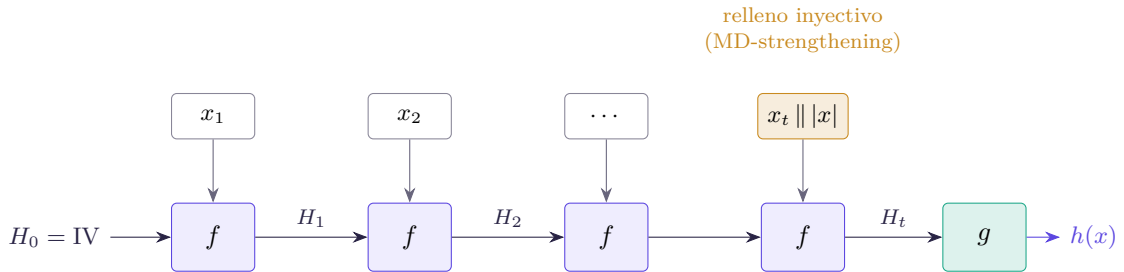


Figura 5: Construcción iterada de Damgård–Merkle. El estado viaja de izquierda a derecha; el digest es el estado final.

Teorema 1 (Damgård–Merkle). Sea $f : \{0, 1\}^n \times \{0, 1\}^b \rightarrow \{0, 1\}^n$ una función de compresión. Tras aplicar un relleno inyectivo que codifica la longitud,

$$H_0 = \text{IV}, \quad H_i = f(H_{i-1}, x_i), \quad h(x) = g(H_t).$$

Si f es resistente a colisiones, entonces h también lo es.

Demostración. Si los mensajes tienen distinto número de bloques, el relleno codifica longitudes distintas en el último bloque, y ahí hay una colisión de f . Si tienen el mismo número, se recorre la cadena hacia atrás: existe un primer índice donde las entradas de f difieren pero las salidas coinciden, es decir, una colisión de f . $\square$

El teorema sólo transfiere colisión

No dice nada sobre el comportamiento de h como oráculo aleatorio, y ahí es exactamente donde aparece la extensión de longitud.

3.1. El ataque de extensión de longitud

Esa arquitectura tiene una consecuencia que casi nadie ve venir: si conoces el digest, **conoces el estado interno** con el que quedó la máquina. Y con el estado en la mano puedes seguir amasando —añadir texto al final del mensaje— sin conocer el principio.

Un servidor autentica peticiones con $t = H(k \| m)$, creyendo que sin la clave nadie puede producir un t válido. El atacante ve m y t , nunca la clave, y aun así puede calcular la etiqueta válida de $m \| \text{relleno} \| z$ para el texto z que se le antoje.

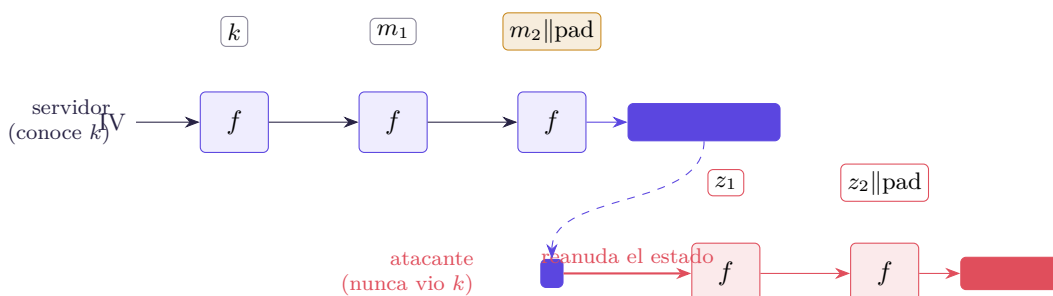


Figura 6: Extensión de longitud. El digest publicado es el estado interno: el atacante retoma la cadena donde el servidor la dejó y autentica un mensaje que nadie firmó.

Caso documentado

En 2009 se falsificaron firmas de la API de Flickr exactamente así, sobre un MAC artesanal de la forma $H(\text{secreto} \| m)$. El error no estaba en SHA-1: estaba en creer que un hash de Damgård–Merkle se comporta como un oráculo aleatorio.

3.2. La construcción correcta: HMAC

$$\text{HMAC}_k(m) = H((k \oplus \text{opad}) \| H((k \oplus \text{ipad}) \| m)) \quad (\text{RFC 2104})$$

El doble anidamiento hace que el estado interno final no sea el de un prefijo del mensaje, y la extensión de longitud muere ahí. Es lo que usan TLS, JWT y la derivación de monederos jerárquicos BIP32 —con HMAC-SHA-512— precisamente por este motivo.

SHA-3 abandona esta arquitectura

Keccak usa una construcción de *esponja* con permutación Keccak- $f[1600]$, absorción a razón r y capacidad $c = 2n$. No sufre extensión de longitud, y esa fue una de las razones de la elección del NIST en 2012. Bitcoin resuelve el mismo problema por otra vía: aplica doble SHA-256 a los encabezados de bloque.

4. El ataque del cumpleaños: por qué 256 bits valen 128

En un salón con 23 personas la probabilidad de que dos compartan cumpleaños supera el 50%. La intuición falla porque uno se pregunta por sí mismo, cuando la pregunta real es por *todos los pares posibles*: con 23 personas hay 253 pares.

Traducido a hashes: no cuesta 2^{256} romper SHA-256 por colisión, cuesta 2^{128} . La raíz cuadrada es el precio de que el atacante pueda elegir ambos mensajes.

4.1. Derivación de la cota

Sea $N = 2^n$ el número de salidas y k el de mensajes hashados al azar. La probabilidad de no colisión es

$$\Pr[\text{sin colisión}] = \prod_{i=0}^{k-1} \frac{N-i}{N}.$$

Usando $1 - t \leq e^{-t}$ con $t = i/N$,

$$\Pr[\text{colisión}] \approx 1 - e^{-k(k-1)/2N}.$$

Imponiendo probabilidad $\frac{1}{2}$ y aproximando $k(k-1) \approx k^2$,

$$k = \sqrt{2 \ln 2} \sqrt{N} \approx 1.1774 \cdot 2^{n/2}. \quad (1)$$

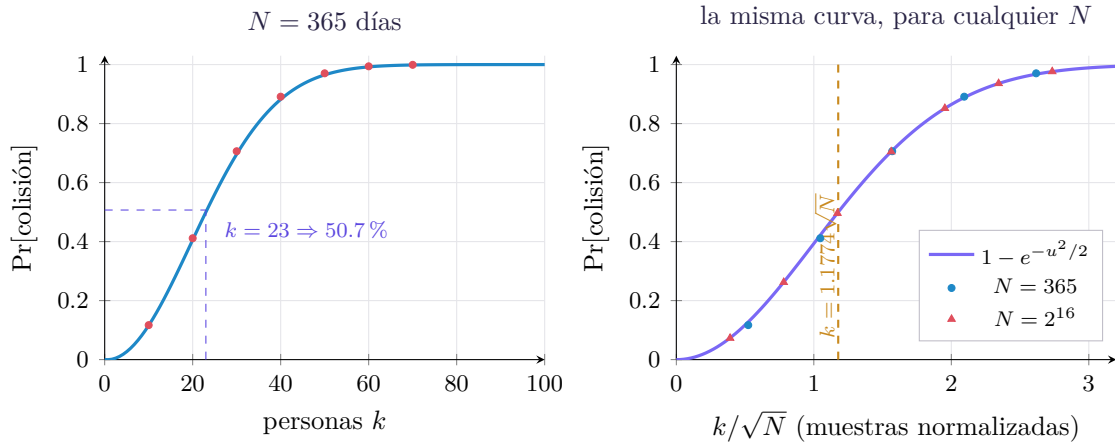


Figura 7: La paradoja del cumpleaños es una sola curva. Al medir las muestras en unidades de $\sqrt{N}$, el salón de clases y un espacio de 2^{16} digests caen sobre la misma gráfica: por eso la ecuación (1) no depende del contexto.

Consecuencia operativa

Una función hash de n bits tiene *seguridad ideal* si hallar una colisión cuesta $\Theta(2^{n/2})$ y hallar una preimagen cuesta $\Theta(2^n)$. Todo ataque publicado que baje de esa cota se considera una **rotura**, aunque siga siendo impracticable. Es la única cifra que permite decidir si una noticia de criptoanálisis importa o no.

Ésta es la razón por la que ningún diseño moderno se conforma con 128 bits de salida: 128 bits de digest son sólo 64 bits de seguridad frente a colisión, y 2^{64} es alcanzable. Es exactamente la historia de MD5, y la figura 8 lo pone en una sola imagen.

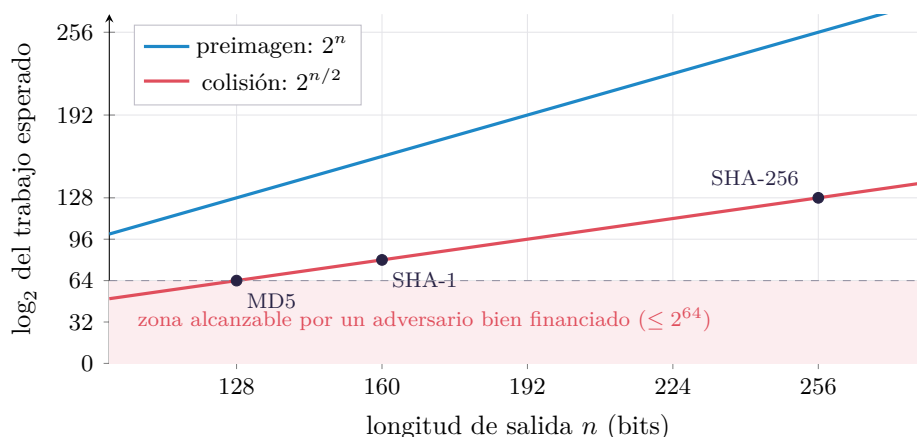


Figura 8: Colisión contra preimagen. La línea de colisión de MD5 nace justo sobre la frontera de lo alcanzable: su ruptura no fue un accidente, era cuestión de tiempo.

Laboratorio 4.1 — La paradoja, en personas y en bits

El mismo cálculo sirve para un salón de clases y para un espacio de digests.

Qué observar. Con 365 días y 23 personas la probabilidad exacta es 50.7 %, y la aproximación exponencial casi no se distingue de ella. Al subir el espacio a 2^{24} hacen falta unas 4 800 muestras, no 8 millones: la raíz cuadrada, otra vez.

5. Cuáles usar hoy, cuáles ya cayeron, y cuándo

Ninguna función hash muere de golpe. Primero alguien publica un ataque que cuesta un poco menos que la fuerza bruta —nadie se alarma—, luego el costo baja lo suficiente para un laboratorio, y un día aparecen dos archivos distintos con el mismo hash, descargables por cualquiera. En ese momento la función ya llevaba años muerta en los papers.

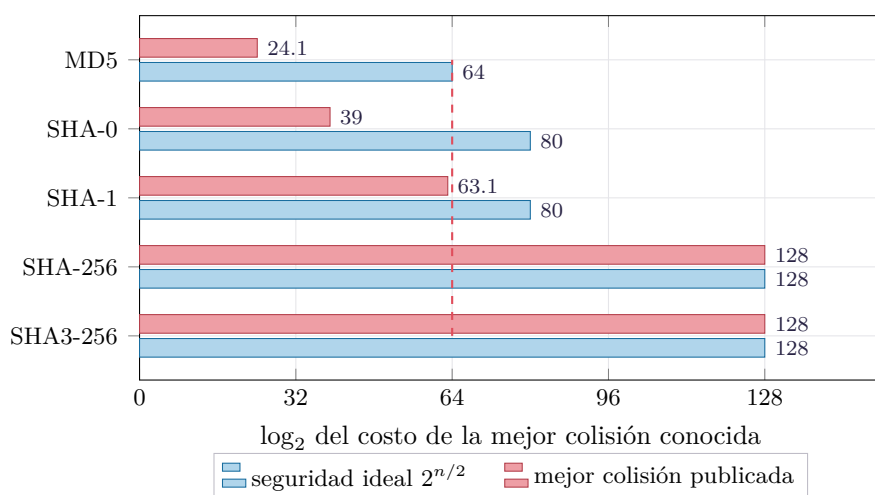


Figura 9: Línea del deterioro. La distancia entre las dos barras es la medida objetiva del daño. SHA-256 y SHA3-256 no tienen rotura conocida, así que sus dos barras coinciden. Estado del arte a agosto de 2026.

Tabla 1: Qué usar, qué no, y por qué. La cifra que importa no es la longitud de la salida, sino la *longitud de seguridad*.

Algoritmo		Seguridad	Mejor ataque conocido	Estado
SHA-256	FIPS 180-4	128 colisión256 preimagen	Cumpleaños genérico. El criptoanálisis dedicado no rebasa 31 de las 64 rondas.	vigente
SHA3-256	Keccak · FIPS 202	128 colisión256 preimagen	No supera 6 de las 24 rondas de Keccak- <i>f</i> . La esponja es inmune a extensión de longitud.	vigente
SHA-1	retirada por el NIST	≈ 63 efectivos	Colisión de prefijo elegido: $2^{63.4}$, ejecutada en 2020 por ≈ 45 000 USD de GPU rentada.	rota
MD5	RFC 1321	< 32 efectivos	Colisión diferencial de Wang: segundos en una laptop. Con prefijo elegido, ≈ 2^{39} .	rota
CRC32	suma de comprobación	0	Invertible en tiempo lineal. Segunda preimagen y colisiones triviales.	no cripto.
Argon2id	PHC 2015 · RFC 9106	parametrizable	Función de derivación de claves: memoria y tiempo configurables. No es para integridad.	contraseñas

Dos precisiones que suelen citarse mal

La cifra 2^{39} corresponde a **SHA-0**, no a SHA-1. Y SHA-3 fue seleccionado en octubre de 2012 y publicado como FIPS 202 en agosto de 2015: no son la misma fecha.

6. Contraseñas: por qué un hash rápido es un defecto

Objetivo del módulo

Entender qué protege la sal, qué protege el factor de trabajo, y por qué SHA-256 es la elección equivocada para almacenar contraseñas.

Guardar SHA-256(contraseña) parece prudente: si roban la base, no hay contraseñas en claro. Pero el atacante no necesita invertir el hash. Le basta con hashear las diez mil contraseñas más comunes y buscar coincidencias —y como el hash es determinista, **una sola tabla sirve para todos los usuarios de todas las bases de datos del mundo**. Eso es una tabla arcoíris.

La *sal* rompe esa reutilización: un valor aleatorio distinto por usuario obliga a rehacer la tabla para cada uno. El *factor de trabajo* ataca lo otro: hace que cada intento cueste 100 ms en lugar de 0.000002 s.

6.1. Los tres escenarios

1. **Ingenuo:** $v = H(p)$. El adversario invierte $|D|$ evaluaciones una sola vez y ataca a u usuarios en $O(|D| + u)$.
2. **Con sal:** $v = H(s \parallel p)$, con s aleatoria pública de al menos 128 bits. El costo pasa a $O(u \cdot |D|)$: la precomputación deja de amortizarse.
3. **Con hash adaptativo:** $v = \text{KDF}(p, s, \tau)$. El defensor paga c por autenticación; el atacante paga $|D| \cdot c$ por usuario:

$$\text{costo}_{\text{atacante}} = u \cdot |D| \cdot c(\tau, m).$$

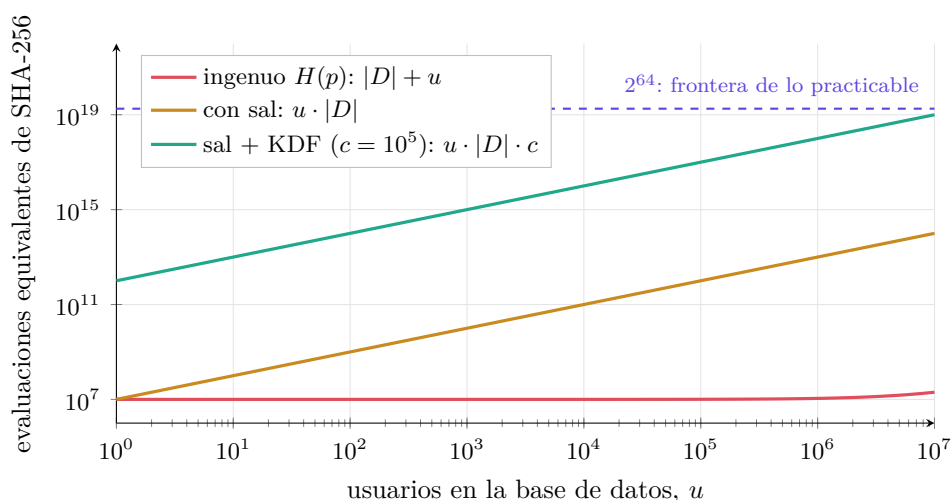


Figura 10: Costo total del ataque por diccionario ($|D| = 10^7$ candidatos). Sin sal, el esfuerzo del adversario es prácticamente constante por más usuarios que haya; la sal lo vuelve lineal, y el factor de trabajo lo empuja cinco órdenes de magnitud hacia arriba.

Caso documentado

LinkedIn, 2012: 6.5 millones de hashes SHA-1 sin sal. La mayoría se recuperó en días. De ahí salieron bcrypt, scrypt y Argon2.

Criterio de diseño

Ajusta τ para que una verificación legítima cueste entre 50 y 250 ms en tu hardware de producción. Ése es el punto en que el usuario no lo nota y el atacante sí.

Laboratorio 6.1 — Tabla arcoíris contra sal

Ocho usuarios ficticios eligen contraseñas de un diccionario común. Se ejecuta el ataque con y sin sal, mirando la columna de *hashes calculados*, no la de cuentas rotas.

Qué observar. La sal no impide adivinar una contraseña débil concreta: impide *atacar a todos a la vez con el mismo trabajo*. Con ocho usuarios el atacante paga ocho veces más; con ocho millones, ocho millones de veces más.

7. Árboles de Merkle: probar pertenencia sin descargar todo

Un bloque de Bitcoin puede contener miles de transacciones. Tu teléfono no va a descargar 600 GB para comprobar que te pagaron.

El árbol de Merkle resuelve exactamente eso: hashea las transacciones por parejas, luego los resultados por parejas, y así hasta quedarse con un único valor —la *raíz*— que resume todo el conjunto.

Definición 2 (Construcción y prueba de inclusión). Dadas m hojas, el nivel base es $L_0 = (\text{dSHA}(T_1), \dots, \text{dSHA}(T_m))$ con $\text{dSHA}(x) = \text{SHA256}(\text{SHA256}(x))$, y recursivamente

$$L_{j+1}[i] = \text{dSHA}(L_j[2i] \parallel L_j[2i+1]),$$

duplicando el último elemento cuando el nivel tiene un número impar de nodos.

Una *prueba de inclusión* para T_i es la lista de hermanos $\pi = (\sigma_0, \dots, \sigma_{d-1})$ con $d = \lceil \log_2 m \rceil$, y el verificador recomputa

$$r_0 = \text{dSHA}(T_i), \quad r_{j+1} = \text{dSHA}(\text{orden}(r_j, \sigma_j)),$$

aceptando si $r_d = \text{raíz}$.

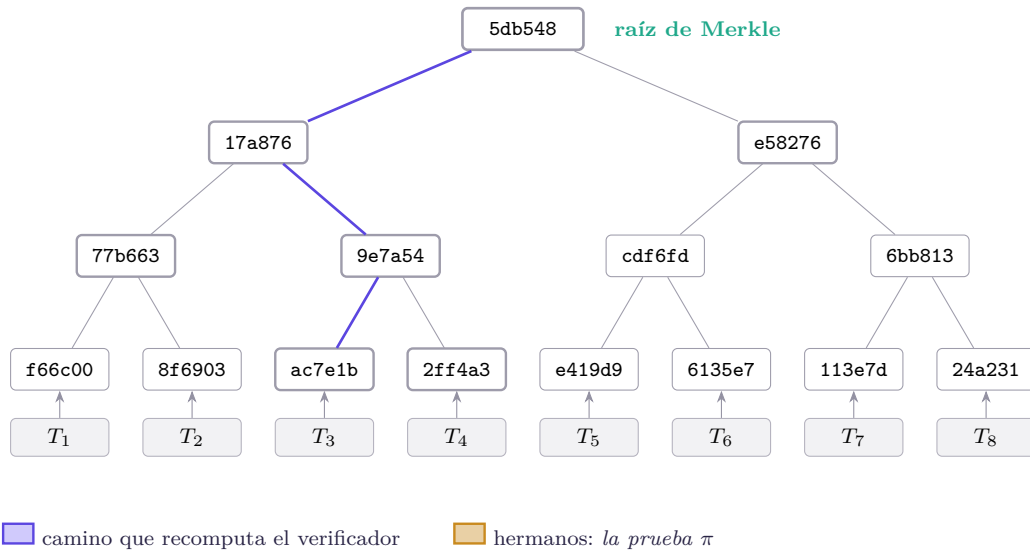


Figura 11: Árbol de Merkle sobre ocho transacciones (doble SHA-256; se muestran los primeros 6 dígitos hexadecimales, calculados realmente). Para convencer a cualquiera de que T_3 está adentro basta con enviar tres hashes hermanos, no el bloque entero.

Lo elegante es la prueba: para convencerte de que tu transacción está adentro, sólo hace falta darte los hashes hermanos del camino hasta la raíz. Con 1 048 576 transacciones son **20 hashes: 640 bytes** en lugar de gigabytes. El ahorro es logarítmico, y por eso la verificación simplificada de pagos (SPV) es posible.

Seguridad

Falsificar una prueba implica hallar una colisión de dSHA en algún nivel. La seguridad del árbol se reduce a la resistencia a colisión de la función base: es el mismo argumento que el del teorema de Damgård–Merkle.

La duplicación de la última hoja

Cuando un nivel tiene un número impar de nodos, el último se duplica. Ésa es la fuente de la vulnerabilidad **CVE-2012-2459** de Bitcoin: dos listas de transacciones distintas podían producir la misma raíz. Buen ejercicio de clase: explicar por qué la duplicación rompe la inyectividad (véase el ejercicio 2 del apéndice).

Laboratorio 7.1 — Constructor de árbol y prueba de inclusión

Se construye el árbol a partir de una lista de transacciones, se pide la prueba de inclusión de una de ellas y después se altera esa misma transacción.

Qué observar. La cascada: un solo bit distinto en la hoja cambia todos los nodos del camino hasta la raíz, y la prueba deja de verificar.

8. Compromisos, prueba de trabajo y la cadena

Un compromiso es un sobre cerrado: pones dentro tu decisión, entregas el sobre, y más tarde lo abres. Quien lo tiene no puede leerlo (*ocultamiento*) y tú ya no puedes cambiar lo que hay dentro (*vinculación*).

Definición 3 (Esquema de compromiso basado en hash). **Compromiso.** El emisor elige m y $r \leftarrow \{0, 1\}^\lambda$ aleatorio, calcula $c = H(m \parallel r)$ y publica c .

Apertura. Revela (m, r) ; el verificador recomputa y compara con c .

Ocultamiento: c no revela información sobre m ; se apoya en la resistencia a preimagen y en que r tenga suficiente entropía.

Vinculación: hallar $(m', r') \neq (m, r)$ con el mismo c equivale a una colisión de H .

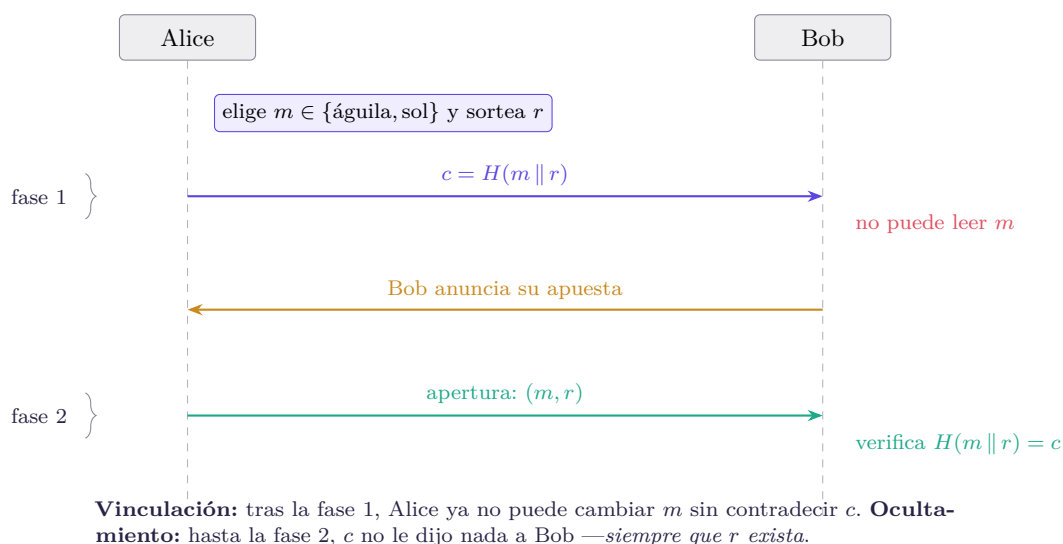


Figura 12: El volado por teléfono. Dos desconocidos deciden algo a distancia sin confiar el uno en el otro, con un hash como único árbitro.

Sin r no hay ocultamiento

Cuando el espacio de m es pequeño, el adversario simplemente hashea todas las opciones. Con dos opciones —águila o sol— le toma dos hashes. El valor aleatorio r no es un adorno: es lo que vuelve impracticable esa búsqueda.

8.1. Minar es resolver una preimagen parcial

La prueba de trabajo pide hallar un *nonce* tal que

$$H(\text{bloque} \parallel \text{nonce}) < \text{objetivo}.$$

Es el único problema difícil de esta lista que se resuelve por fuerza bruta *a propósito*: la dificultad es el producto, no el defecto. Y como cada bloque incluye el hash del anterior, alterar uno invalida todos los siguientes.



La cascada. Edita el contenido del bloque 1 y su hash deja de empezar con 000: el bloque 1 se invalida, el campo *prev* del bloque 2 ya no coincide, y el 3 tampoco. Para reescribir la historia hay que volver a minar ese bloque *y todos los posteriores*. Eso es el costo de mentir.

Figura 13: Tres bloques minados de verdad con dificultad de 3 ceros hexadecimales. Cada cero adicional multiplica por 16 el trabajo esperado; la red de Bitcoin opera hoy alrededor de 19 ceros.

Por qué SHA-256 se ejecuta en masa

Un equipo comercial de minado ronda los 580 TH/s: en un segundo calcula lo que un núcleo de tu procesador tardaría más de medio año. SHA-256 es el sueño de un diseñador de circuitos —64 rondas fijas, sin memoria ni bifurcaciones— y por eso un ASIC entrega un hash por ciclo de reloj. Las funciones *memory-hard* como Argon2 o RandomX invierten el argumento: si el cuello de botella es la memoria y no la aritmética, el fabricante de silicio pierde su ventaja, porque la DRAM se la vende el mismo proveedor a todo el mundo.

Laboratorios 8.1 y 8.2 — Volado por teléfono; prueba de trabajo

En el primero se ejecuta el protocolo de compromiso y luego se repite quitando r : Bob rompe el ocultamiento al instante. En el segundo se minan tres bloques y se edita el contenido de uno para ver la cascada de invalidación.

Qué observar. Cada cero hexadecimal adicional multiplica por 16 el trabajo esperado. Por eso el ajuste de dificultad mide trabajo real y no buena suerte.

9. Síntesis: qué se rompe exactamente cuando falla cada propiedad

Hasta aquí las propiedades se enunciaron por separado. Conviene ahora invertir la pregunta: si mañana alguien rompe la resistencia a colisión de la función que estás usando, ¿qué deja de

funcionar y qué sigue en pie? La respuesta no es «todo».

El caso canónico: la firma digital

Nadie firma un documento: **se firma su hash**. Si el atacante consigue dos textos con la misma huella —uno inocuo y otro fraudulento— basta con hacer firmar el primero para quedarse con una firma válida sobre el segundo. La clave privada nunca se compromete y, aun así, la firma es falsificable. Ésa es la consecuencia exacta de perder la resistencia a colisión, y es lo que ocurrió con el certificado de CA falso construido sobre MD5 en 2008 y con el malware Flame en 2012.

Tabla 2: Mapa de consecuencias. Una propiedad rota no derriba el sistema entero: derriba los elementos cuya seguridad se reduce a ella.

Propiedad	Si falla, el adversario puede...	Dónde duele primero	Costo ideal
Preimagen one-way	Invertir huellas publicadas: recuperar contraseñas de un volcado, leer el contenido de un compromiso antes de que lo abran.	Ocultamiento del compromiso · contraseñas · direcciones	2^n
2.^a preimagen target collision	Sustituir un objeto ya publicado y aceptado por otro sin alterar ningún hash previamente difundido.	Encadenamiento de bloques · prueba de Merkle · documento firmado	2^n
Colisión collision-free	Fabricar de antemano dos objetos intercambiables: hacer firmar uno y exhibir el otro, o abrir un mismo compromiso de dos maneras.	Firma digital · vinculación del compromiso · raíz de Merkle	$2^{n/2}$
Colisión de prefijo elegido chosen-prefix	Dar a las dos versiones un encabezado realista fijado de antemano —nombres, montos, campos de un certificado X.509—, no dos archivos basura.	Documentos con formato · certificados · PGP	$\geq 2^{n/2}$
Inmunidad a extensión de longitud propiedad de la construcción	Forjar etiquetas de autenticación para mensajes que nunca vio, sin conocer la clave, si el MAC es el ingenuo $H(k \parallel m)$.	MAC artesanal · APIs firmadas · BIP32	—
Salida indistinguible de aleatoria avalancha · uniformidad	Sesgar o anticipar valores derivados del hash: el nonce ganador, el resultado de un sorteo, la clave hija de un monedero jerárquico.	Prueba de trabajo · aleatoriedad pública · HMAC	—
Costo de evaluación controlable sólo para KDF	Evaluar miles de millones de candidatos por segundo en GPU o ASIC: la velocidad, virtud en todo lo demás, aquí es el defecto.	Almacenamiento de contraseñas · cifrado de monederos	—

9.1. Una nota sobre el adversario cuántico

El algoritmo de Grover reduce la búsqueda de preimagen de 2^n a $\approx 2^{n/2}$, y el de Brassard–Høyer–Tapp llevaría las colisiones a $2^{n/3}$ —pero exigiendo memoria cuántica de acceso aleatorio

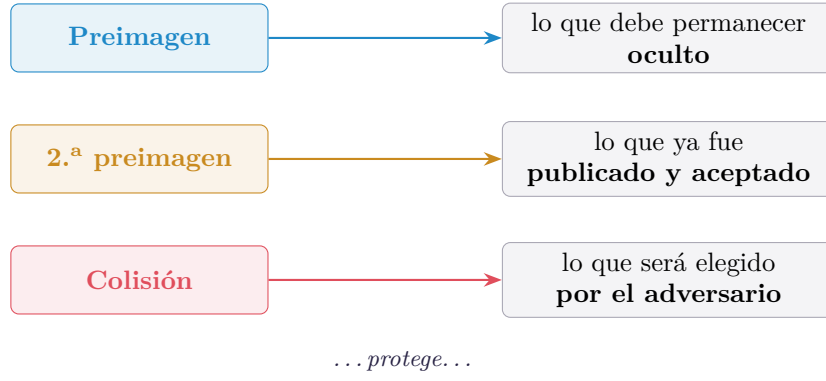


Figura 14: La regla que ordena la tabla 2: cada resistencia custodia una clase distinta de objeto.

en una escala que hoy nadie considera realista. Bajo modelos con memoria acotada, la ventaja cuántica sobre colisiones casi desaparece.

La consecuencia práctica es sencilla: SHA-256 conserva del orden de 128 bits frente a preimagen cuántica y **no es la pieza urgente**. La presión post-cuántica está en la firma —ECDSA cae ante Shor—, no en la función hash. De hecho, los esquemas de firma construidos exclusivamente con hash (FIPS 205, SLH-DSA) son la respuesta, no el problema.

Proposición 3 (hash-then-sign). *Sea Σ un esquema de firma seguro frente a falsificación existencial, y Σ^H el esquema que firma $H(m)$ en lugar de m . Para todo falsificador $\mathcal{A}$ existen $\mathcal{B}, \mathcal{C}$ eficientes tales que*

$$\text{Adv}_{\Sigma^H}^{\text{euf-cma}}(\mathcal{A}) \leq \text{Adv}_{\Sigma}^{\text{euf-cma}}(\mathcal{B}) + \text{Adv}_H^{\text{col}}(\mathcal{C}).$$

Observación. La lectura importante es la contrapuesta. La desigualdad sólo garantiza seguridad mientras el segundo término sea despreciable; en cuanto la ventaja de colisión deja de serlo, la cota se vacía y la falsificación es efectiva. Eso es exactamente lo que ocurrió con MD5 y con SHA-1.

A. Resumen de cotas

Tabla 3: Todo el módulo en nueve renglones.

Cantidad	Expresión	Para $n = 256$
Preimagen (clásica)	2^n	2^{256}
Segunda preimagen	2^n	2^{256}
Colisión (cumpleaños)	$\approx 1.1774 \cdot 2^{n/2}$	$\approx 2^{128}$
Preimagen (Grover)	$\approx 2^{n/2}$	$\approx 2^{128}$
Colisión (BHT, RAM cuántica)	$\approx 2^{n/3}$	$\approx 2^{85}$
Avalancha esperada	$\mathbb{E}[d_H] = n/2$	128 bits
Desviación de la avalancha	$\sigma = \sqrt{n}/2$	8 bits
Tamaño de prueba de Merkle	$\lceil \log_2 m \rceil$ hashes	20 hashes para $m = 2^{20}$
Costo del diccionario con sal	$u \cdot D \cdot c$	lineal en usuarios

B. Ejercicios propuestos

1. Demuestra que la función h de la Proposición 2 es también resistente a segunda preimagen para entradas de longitud $\neq n$, y explica por qué eso no contradice nada de lo dicho.
2. **CVE-2012-2459.** Sea un árbol de Merkle sobre la lista (T_1, T_2, T_3) , que duplica la última hoja para completar el nivel. Exhibe una lista distinta de cuatro transacciones con exactamente la misma raíz, y argumenta por qué el atacante no necesita ninguna colisión de SHA-256 para construirla.
3. Un servidor autentica con $t = H(m \parallel k)$ —la clave *al final*—. ¿Siguió funcionando el ataque de extensión de longitud? ¿Qué otro problema aparece en su lugar cuando H pierde la resistencia a colisión?
4. Usando la ecuación (1), calcula cuántas direcciones distintas hay que generar para que la probabilidad de que dos coincidan alcance el 1% con $n = 160$ bits. Compara con el número de direcciones que ha visto la red de Bitcoin en toda su historia.
5. Un sistema guarda $v = H(s \parallel p)$ con sal de 128 bits pero usa SHA-256 de una sola pasada. Cuantifica, con la figura 10, cuánto gana el defensor al migrar a Argon2id con $c = 10^5$, y explica por qué la sal por sí sola no bastaba.
6. Demuestra que un compromiso $c = H(m)$ sin valor aleatorio sigue siendo vinculante pero pierde por completo el ocultamiento cuando $|\mathcal{M}| = 2$. ¿Cuántos hashes necesita el adversario?